

# 1 程序基础

## 1.1 最小程序结构

```
#include <stdio.h>          /* 预处理：包含头文件 */

int main(void) {            /* 主函数，程序入口 */
    printf("Hello, C!\n");
    return 0;               /* 返回 0 表示正常结束 */
}
```

## 1.2 编译与运行流程

| 阶段  | 说明                         | 常见产物            |
|-----|----------------------------|-----------------|
| 预处理 | 处理 # 指令 (include、define 等) | .i 文件           |
| 编译  | 将 C 代码翻译为汇编                | .s 文件           |
| 汇编  | 汇编代码转为机器码                  | .o / .obj 目标文件  |
| 链接  | 合并目标文件与库，生成可执行文件           | .exe / 无后缀可执行文件 |

常见命令 (GCC)：

| 命令                         | 作用               |
|----------------------------|------------------|
| gcc hello.c -o hello       | 编译并指定输出名         |
| gcc -Wall hello.c -o hello | 开启常见警告           |
| gcc -c hello.c             | 只编译不链接，生成 .o     |
| ./hello                    | 运行 (Linux/macOS) |
| hello.exe                  | 运行 (Windows)     |

## 1.3 源文件与头文件

| 类型  | 扩展名             | 作用                                  |
|-----|-----------------|-------------------------------------|
| 源文件 | <code>.c</code> | 存放函数实现                              |
| 头文件 | <code>.h</code> | 存放声明、宏、类型定义，供 <code>#include</code> |

## 1.4 注释写法

| 方式 | 语法                        | 说明      |
|----|---------------------------|---------|
| 单行 | <code>// 注释</code>        | C99 起支持 |
| 单行 | <code>/* 注释 */</code>     | 传统写法    |
| 多行 | <code>/* 多行\n注释 */</code> | 不能嵌套    |

## 1.5 标识符命名规则

| 规则     | 说明                                       | 合法示例                                   | 非法示例              |
|--------|------------------------------------------|----------------------------------------|-------------------|
| 首字符    | 字母或 <code>_</code>                       | <code>_count</code> , <code>sum</code> | <code>2abc</code> |
| 后续字符   | 字母、数字、 <code>_</code>                    | <code>var_1</code>                     | <code>a-b</code>  |
| 区分大小写  | <code>Sum</code> 与 <code>sum</code> 不同   | —                                      | —                 |
| 不能是关键字 | 如 <code>int</code> 、 <code>return</code> | <code>my_int</code>                    | <code>int</code>  |

## 1.6 C 语言关键字（常用）

| 类别    | 关键字                                                                                                                                                                                                         |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 类型    | <code>char</code> <code>int</code> <code>float</code> <code>double</code> <code>void</code> <code>short</code> <code>long</code> <code>signed</code> <code>unsigned</code>                                  |
| 控制流   | <code>if</code> <code>else</code> <code>switch</code> <code>case</code> <code>default</code> <code>for</code> <code>while</code> <code>do</code> <code>break</code> <code>continue</code> <code>goto</code> |
| 函数/存储 | <code>return</code> <code>static</code> <code>extern</code> <code>register</code> <code>auto</code> <code>const</code> <code>volatile</code>                                                                |
| 结构    | <code>struct</code> <code>union</code> <code>enum</code> <code>typedef</code>                                                                                                                               |
| 其他    | <code>sizeof</code> <code>break</code> <code>continue</code>                                                                                                                                                |

## 1.7 语句与表达式

| 概念   | 说明                        | 示例                                               |
|------|---------------------------|--------------------------------------------------|
| 表达式  | 有值的式子                     | <code>a + 1</code> 、 <code>x &gt; 0</code>       |
| 语句   | 以 <code>;</code> 结尾，执行动作  | <code>a = 5;</code> 、 <code>printf("hi");</code> |
| 复合语句 | 用 <code>{ }</code> 包裹多条语句 | <code>if (x) { a=1; b=2; }</code>                |

## 1.8 C语言主要特点（速记）

| 特点      | 说明                        |
|---------|---------------------------|
| 面向过程    | 以函数为组织单位                  |
| 编译型     | 先编译再运行，效率高                |
| 贴近硬件    | 可指针操作、位运算                 |
| 可移植     | 标准 C 代码跨平台性好              |
| 无自动内存管理 | 动态内存需手动 <code>free</code> |
| 弱类型检查   | 数组不检查边界，需程序员注意            |

# 2 数据类型与常量

## 2.1 基本数据类型

| 类型                     | 含义     | 典型大小（32/64位） | 格式符                                                 |
|------------------------|--------|--------------|-----------------------------------------------------|
| <code>char</code>      | 字符/小整数 | 1 字节         | <code>%c</code>                                     |
| <code>short</code>     | 短整型    | 2 字节         | <code>%hd</code>                                    |
| <code>int</code>       | 整型     | 4 字节         | <code>%d</code>                                     |
| <code>long</code>      | 长整型    | 4/8 字节       | <code>%ld</code>                                    |
| <code>long long</code> | 更长整型   | 8 字节         | <code>%lld</code>                                   |
| <code>float</code>     | 单精度浮点  | 4 字节         | <code>%f</code>                                     |
| <code>double</code>    | 双精度浮点  | 8 字节         | <code>%lf</code> （scanf） / <code>%f</code> （printf） |
| <code>void</code>      | 无类型    | —            | 用于函数返回值或通用指针                                        |

## 2.2 类型修饰符

| 修饰符                   | 作用         | 示例                                |
|-----------------------|------------|-----------------------------------|
| <code>signed</code>   | 有符号（默认）    | <code>signed int x;</code>        |
| <code>unsigned</code> | 无符号，只表示非负数 | <code>unsigned int n;</code>      |
| <code>short</code>    | 缩短整型       | <code>short int s;</code>         |
| <code>long</code>     | 加长整型       | <code>long int l;</code>          |
| <code>const</code>    | 只读，不可修改    | <code>const int MAX = 100;</code> |

## 2.3 变量声明与初始化

| 写法    | 说明        | 示例                             |
|-------|-----------|--------------------------------|
| 声明    | 指定类型和名字   | <code>int age;</code>          |
| 初始化   | 定义时赋初值    | <code>int x = 10;</code>       |
| 多变量   | 同类型一次声明   | <code>int a = 1, b = 2;</code> |
| 局部变量  | 未初始化值不确定  | <code>int x;</code>            |
| 全局/静态 | 未初始化默认为 0 | <code>static int count;</code> |

## 2.4 整型常量写法

| 进制    | 前缀                                | 示例                | 实际值 |
|-------|-----------------------------------|-------------------|-----|
| 十进制   | 无                                 | <code>255</code>  | 255 |
| 八进制   | <code>0</code>                    | <code>0377</code> | 255 |
| 十六进制  | <code>0x</code> / <code>0X</code> | <code>0xFF</code> | 255 |
| 长整型后缀 | <code>L</code> / <code>l</code>   | <code>100L</code> | —   |
| 无符号后缀 | <code>U</code> / <code>u</code>   | <code>100U</code> | —   |

十进制：25(2\*10+5)

二进制：0=0 1=1 2=10 3=11(12+1) 4=100(12^2+0\*2^1+0) 5=101(4+0+1) 7=111 (4+2+1)

2^0 = 1

2^1 = 2

2^2 = 4

$2^3 = 8$

$2^4 = 16$

十进制：0 1 2 3 4 5 6 7 8 9

二进制：0 1

八进制：0 1 2 3 4 5 6 7

十六进制：0 1 2 3 4 5 6 7 8 9 A B C D E F

方法：短除法（进制转换）

## 2.5 浮点常量

| 写法     | 说明           | 示例              |
|--------|--------------|-----------------|
| 小数形式   | 含小数点         | 3.14            |
| 指数形式   | e/E 表示 10 的幂 | 1.2e-3 → 0.0012 |
| 单精度后缀  | f / F        | 3.14f           |
| 长双精度后缀 | L            | 3.14L           |

## 2.6 字符与字符串常量

| 类型  | 定界符     | 示例       | 说明         |
|-----|---------|----------|------------|
| 字符  | 单引号 ' ' | 'A'、'\n' | 占 1 字节（存储） |
| 字符串 | 双引号 " " | "Hello"  | 末尾自动加 \0   |

## 2.7 常用转义字符

| 转义                | 含义           |
|-------------------|--------------|
| <code>\n</code>   | 换行           |
| <code>\t</code>   | 制表符          |
| <code>\\</code>   | 反斜杠          |
| <code>\'</code>   | 单引号          |
| <code>\"</code>   | 双引号          |
| <code>\0</code>   | 空字符（字符串结束）   |
| <code>\ddd</code> | 八进制 ASCII 码  |
| <code>\xhh</code> | 十六进制 ASCII 码 |

## 2.8 常量定义方式

| 方式       | 语法                                      | 特点        |
|----------|-----------------------------------------|-----------|
| 宏常量      | <code>#define PI 3.14</code>            | 预处理替换，无类型 |
| const 变量 | <code>const double PI = 3.14;</code>    | 有类型，更安全   |
| 枚举       | <code>enum { RED, GREEN, BLUE };</code> | 整型常量集     |

## 2.9 sizeof 求类型/变量大小

| 用法 | 示例                        | 结果（常见）      |
|----|---------------------------|-------------|
| 类型 | <code>sizeof(int)</code>  | 4           |
| 变量 | <code>sizeof(x)</code>    | 取决于 x 的类型   |
| 字符 | <code>sizeof(char)</code> | 1           |
| 数组 | <code>sizeof(a)</code>    | 元素个数 × 元素大小 |

## 2.10 类型转换

| 类型   | 说明                    | 示例                                            |
|------|-----------------------|-----------------------------------------------|
| 隐式转换 | 编译器自动转换               | <code>int x = 3.9;</code> → <code>x=3</code>  |
| 显式转换 | 强制类型转换                | <code>(int)3.9</code> 、 <code>(float)n</code> |
| 整型提升 | char/short 运算时提升为 int | <code>'a' + 1</code>                          |

## 2.11 取值范围（典型，有符号 int）

| 类型                        | 约略范围                            |
|---------------------------|---------------------------------|
| <code>char</code>         | -128 ~ 127 或 0 ~ 255 (unsigned) |
| <code>short</code>        | -32768 ~ 32767                  |
| <code>int</code>          | 约 ±21 亿                         |
| <code>unsigned int</code> | 0 ~ 约 42 亿                      |

# 3 运算符

## 3.1 算术运算符

| 运算符            | 含义 | 示例                 | 注意             |
|----------------|----|--------------------|----------------|
| <code>+</code> | 加  | <code>a + b</code> | —              |
| <code>-</code> | 减  | <code>a - b</code> | —              |
| <code>*</code> | 乘  | <code>a * b</code> | —              |
| <code>/</code> | 除  | <code>a / b</code> | 整数/整数 → 整数（截断） |
| <code>%</code> | 取模 | <code>a % b</code> | 仅用于整型          |

整数除法示例：`5 / 2 = 2`，`5.0 / 2 = 2.5`

## 3.2 关系运算符

| 运算符       | 含义   | 结果类型       |
|-----------|------|------------|
| > < >= <= | 比较大小 | 真为 1，假为 0  |
| ==        | 等于   | 注意与赋值 = 区分 |
| !=        | 不等于  | —          |

### 3.3 逻辑运算符

| 运算符 | 含义  | 示例     | 短路     |
|-----|-----|--------|--------|
| &&  | 逻辑与 | a && b | 左假则不算右 |
|     | 逻辑或 | a    b | —      |
| !   | 逻辑非 | !flag  | —      |

### 3.4 赋值运算符

| 运算符 | 等价于       | 示例     |
|-----|-----------|--------|
| =   | —         | a = 10 |
| +=  | a = a + b | a += 5 |
| -=  | a = a - b | a -= 3 |
| *=  | a = a * b | a *= 2 |
| /=  | a = a / b | a /= 2 |
| %=  | a = a % b | a %= 3 |

**结合性：** 从右到左，如 `a = b = 5` 先给 b 赋 5，再给 a 赋 5。

### 3.5 自增自减

| 写法        | 含义     | 表达式值  |
|-----------|--------|-------|
| ++i       | i 先加 1 | 加之后的值 |
| i++       | i 后加 1 | 加之前的值 |
| --i / i-- | 同理减 1  | 同理    |

### 3.6 位运算符

| 运算符 | 含义   | 示例            |
|-----|------|---------------|
| &   | 按位与  | a & b         |
| 、   | 、    | 按位或           |
| ^   | 按位异或 | a ^ b         |
| ~   | 按位取反 | ~a            |
| <<  | 左移   | a << 2 相当于 ×4 |
| >>  | 右移   | a >> 1 相当于 ÷2 |

### 3.7 其他运算符

| 运算符    | 名称     | 示例                    |
|--------|--------|-----------------------|
| ? :    | 条件运算符  | max = a > b ? a : b   |
| ,      | 逗号     | x = 1, y = 2 (值为最后一个) |
| &      | 取地址    | &x                    |
| *      | 解引用    | *p                    |
| sizeof | 求大小    | sizeof(int)           |
| .      | 结构体成员  | stu.name              |
| ->     | 指针访问成员 | p->name               |

### 3.8 运算符优先级（从高到低，常用部分）

| 优先级 | 运算符                                   | 结合性            |
|-----|---------------------------------------|----------------|
| 1   | <code>() [] -&gt; .</code>            | 左→右            |
| 2   | <code>! ~ ++ -- * &amp; sizeof</code> | 右→左            |
| 3   | <code>* / %</code>                    | 左→右            |
| 4   | <code>+ -</code>                      | 左→右            |
| 5   | <code>&lt;&lt; &gt;&gt;</code>        | 左→右            |
| 6   | <code>&lt; &lt;= &gt; &gt;=</code>    | 左→右            |
| 7   | <code>== !=</code>                    | 左→右            |
| 8   | <code>&amp;</code>                    | 左→右            |
| 9   | <code>^</code>                        | 左→右            |
| 10  | <code>`</code>                        | <code>`</code> |
| 11  | <code>&amp;&amp;</code>               | 左→右            |
| 12  | <code>`</code>                        |                |
| 13  | <code>?:</code>                       | 右→左            |
| 14  | <code>= += -= ...</code>              | 右→左            |
| 15  | <code>,</code>                        | 左→右            |

记忆口诀：单目 → 算术 → 移位 → 关系 → 位运算 → 逻辑 → 条件 → 赋值 → 逗号

### 3.9 常见表达式写法

| 场景          | 写法                                                |
|-------------|---------------------------------------------------|
| 交换两数（需临时变量） | <code>t=a; a=b; b=t;</code>                       |
| 求绝对值        | <code>x = x &lt; 0 ? -x : x</code>                |
| 判断奇偶        | <code>n % 2 == 0</code>                           |
| 判断闰年        | <code>!(y%4==0 &amp;&amp; y%100!=0)</code>        |
| 判断字符是数字     | <code>ch &gt;= '0' &amp;&amp; ch &lt;= '9'</code> |

## 4 流程控制

# 4.1 if 语句

| 形式   | 语法                                   | 说明                       |
|------|--------------------------------------|--------------------------|
| 单分支  | <code>if (条件) 语句;</code>             | 条件为真执行                   |
| 双分支  | <code>if (条件) 语句1; else 语句2;</code>  | 二选一                      |
| 多分支  | <code>if ... else if ... else</code> | 多条件                      |
| 复合语句 | <code>if (条件) { ... }</code>         | 多条语句必须加 <code>{ }</code> |

```
if (score >= 90)
    grade = 'A';
else if (score >= 60)
    grade = 'B';
else
    grade = 'F';
```

# 4.2 switch 语句

| 要点      | 说明                 |
|---------|--------------------|
| 表达式     | 必须是整型或枚举           |
| case    | 后接整型常量表达式          |
| break   | 通常每个 case 末尾加，防止贯穿 |
| default | 可选，所有 case 不匹配时执行  |

```
switch (op) {
    case '+': result = a + b; break;
    case '-': result = a - b; break;
    default: printf("无效\n"); break;
}
```

# 4.3 for 循环

| 部分  | 作用           |
|-----|--------------|
| 初始化 | 循环变量初值，只执行一次 |
| 条件  | 每次循环前判断，假则退出 |
| 更新  | 每次循环体后执行     |

```
for (int i = 0; i < 10; i++)
    printf("%d ", i);
```

## 4.4 while 循环

```
while (条件) {
    /* 循环体 */
}
```

**特点：**先判断后执行，条件一开始为假则一次都不执行。

```
int n = 100;

while (n < 50){

//....

n = n -2;

}

int n = n + 100;
```

## 4.5 do-while 循环

```
do {
    /* 循环体 */
} while (条件); /* 注意末尾分号 */
```

**特点：**先执行后判断，至少执行一次。

## 4.6 三种循环对比

| 循环       | 判断时机 | 最少执行次数 | 典型场景        |
|----------|------|--------|-------------|
| for      | 前    | 0      | 已知次数        |
| while    | 前    | 0      | 未知次数、条件驱动   |
| do-while | 后    | 1      | 至少执行一次（如菜单） |

## 4.7 break 与 continue

| 语句                    | 作用                      | 适用                        |
|-----------------------|-------------------------|---------------------------|
| <code>break</code>    | 跳出 <b>当前</b> 循环或 switch | for/while/do-while/switch |
| <code>continue</code> | 跳过本次循环剩余语句，进入下一次        | for/while/do-while        |

## 4.8 goto（了解即可）

```
goto label;
/* ... */
label:
    /* 跳转目标 */
```

**建议：**初学少用，优先用结构化控制语句。

## 4.9 常见循环模式

| 场景       | 写法                                               |
|----------|--------------------------------------------------|
| 累加 1~n   | <code>for (i=1; i&lt;=n; i++) sum += i;</code>   |
| 遍历数组     | <code>for (i=0; i&lt;n; i++) ... a[i] ...</code> |
| 读直到 EOF  | <code>while ((ch=getchar()) != EOF) ...</code>   |
| 无限循环     | <code>while (1) { ... if (条件) break; }</code>    |
| 嵌套循环打印图案 | 外层行，内层列                                          |

## 4.10 选择结构速查

| 需求     | 推荐                                                    |
|--------|-------------------------------------------------------|
| 两路分支   | <code>if-else</code>                                  |
| 多路等值分支 | <code>switch</code>                                   |
| 范围判断   | <code>if-else if</code> 链                             |
| 多条件组合  | <code>if (a &amp;&amp; b)</code> 、 <code>`if(a</code> |

# 5 函数

## 5.1 函数基本结构

```
返回类型 函数名(参数列表) {  
    /* 函数体 */  
    return 返回值;    /* void 函数可省略或写 return; */  
}
```

## 5.2 函数声明与定义

| 概念     | 说明        | 示例                                  |
|--------|-----------|-------------------------------------|
| 声明（原型） | 告诉编译器函数存在 | <code>int max(int a, int b);</code> |
| 定义     | 函数的具体实现   | 含函数体                                |
| 调用     | 使用函数      | <code>int m = max(3, 5);</code>     |

## 5.3 参数传递

| 方式   | 语法                                  | 特点        |
|------|-------------------------------------|-----------|
| 值传递  | <code>void f(int x)</code>          | 修改形参不影响实参 |
| 地址传递 | <code>void f(int *x)</code>         | 可通过指针修改实参 |
| 数组传递 | <code>void f(int a[], int n)</code> | 实际传首元素地址  |

```
void swap(int *a, int *b) {  
    int t = *a; *a = *b; *b = t;  
}  
/* 调用: swap(&x, &y); */
```

## 5.4 返回值

| 返回类型                                     | 说明                               |
|------------------------------------------|----------------------------------|
| <code>int</code> 、 <code>double</code> 等 | <code>return 表达式;</code>         |
| <code>void</code>                        | 无返回值, <code>return;</code> 或直接结束 |
| 指针                                       | 可返回指针, <b>不要返回局部变量地址</b>         |

## 5.5 递归函数

| 要素   | 说明          |
|------|-------------|
| 终止条件 | 必须存在，否则无限递归 |
| 递归关系 | 大问题化为小问题    |

```
int fact(int n) {
    if (n <= 1) return 1;          /* 终止条件 */
    return n * fact(n - 1);        /* 递归 */
}
```

## 5.6 存储类型

| 关键字                      | 作用             | 作用域 | 生命周期   |
|--------------------------|----------------|-----|--------|
| <code>auto</code>        | 局部变量默认         | 块内  | 函数调用期间 |
| <code>static</code> (局部) | 保留值，不重新初始化     | 块内  | 整个程序   |
| <code>static</code> (全局) | 仅本文件可见         | 文件内 | 整个程序   |
| <code>extern</code>      | 声明外部定义的变量/函数   | 跨文件 | —      |
| <code>register</code>    | 建议放寄存器（编译器可忽略） | 块内  | 函数调用期间 |

## 5.7 函数分类速查

| 类型     | 示例                                             |
|--------|------------------------------------------------|
| 无参无返回值 | <code>void menu(void);</code>                  |
| 有参有返回值 | <code>int add(int a, int b);</code>            |
| 数组作参数  | <code>void sort(int a[], int n);</code>        |
| 字符串作参数 | <code>int len(char s[]);</code>                |
| 函数指针参数 | <code>void qsort(..., int (*cmp)(...));</code> |

## 5.8 main 函数

| 形式                                            | 说明                                    |
|-----------------------------------------------|---------------------------------------|
| <code>int main(void)</code>                   | 无命令行参数                                |
| <code>int main(int argc, char *argv[])</code> | 接收命令行参数                               |
| 返回值                                           | <code>return 0;</code> 表示成功, 非 0 表示异常 |

## 5.9 头文件与多文件

| 文件                 | 内容                            |
|--------------------|-------------------------------|
| <code>xxx.h</code> | 函数声明、宏、类型定义                   |
| <code>xxx.c</code> | 函数实现                          |
| 使用                 | <code>#include "xxx.h"</code> |

示例:

```
/* math_utils.h */
int add(int a, int b);

/* math_utils.c */
int add(int a, int b) { return a + b; }

/* main.c */
#include "math_utils.h"
int main(void) { printf("%d\n", add(1, 2)); return 0; }
```

## 5.10 内联函数 (C99)

```
inline int square(int x) { return x * x; }
```

编译器可能将函数体直接展开, 减少调用开销。

# 6 指针

## 6.1 指针概念

| 概念                     | 说明         |
|------------------------|------------|
| 地址                     | 内存中每个字节的编号 |
| 指针                     | 存放地址的变量    |
| 取地址 <code>&amp;</code> | 获取变量地址     |
| 解引用 <code>*</code>     | 通过指针访问所指对象 |

## 6.2 指针声明与使用

| 写法                          | 含义            |
|-----------------------------|---------------|
| <code>int *p;</code>        | p 是指向 int 的指针 |
| <code>p = &amp;x;</code>    | p 指向 x        |
| <code>*p = 10;</code>       | 通过 p 修改 x 的值  |
| <code>int *p = NULL;</code> | 空指针，不指向有效对象   |

```
int x = 5;
int *p = &x;
printf("%d\n", *p); /* 输出 5 */
*p = 10;           /* x 变为 10 */
```

## 6.3 指针与数组

| 关系                | 说明                                         |
|-------------------|--------------------------------------------|
| 数组名               | 多数情况下等价于首元素地址                              |
| <code>a[i]</code> | 等价于 <code>*(a + i)</code>                  |
| <code>p[i]</code> | 等价于 <code>*(p + i)</code>                  |
| 指针移动              | <code>p+1</code> 移动一个 <b>元素</b> 大小，不是 1 字节 |

## 6.4 指针运算

| 运算                    | 说明             |
|-----------------------|----------------|
| <code>p + n</code>    | 向后移 n 个元素      |
| <code>p - n</code>    | 向前移 n 个元素      |
| <code>p1 - p2</code>  | 两指针间元素个数（同数组内） |
| <code>p1 == p2</code> | 比较地址是否相同       |

## 6.5 常见指针类型

| 类型                          | 说明          |
|-----------------------------|-------------|
| <code>int *p</code>         | 指向 int      |
| <code>char *p</code>        | 指向字符，常用于字符串 |
| <code>void *p</code>        | 通用指针，需强转后使用 |
| <code>int **pp</code>       | 指向指针的指针     |
| <code>int (*fp)(int)</code> | 函数指针        |

## 6.6 指针作为函数参数

| 用途     | 示例                                      |
|--------|-----------------------------------------|
| 修改实参   | <code>void swap(int *a, int *b)</code>  |
| 传递数组   | <code>void func(int *arr, int n)</code> |
| 返回多个结果 | 通过指针参数写回                                |
| 传递大结构体 | 传地址避免复制                                 |

## 6.7 指针与 const

| 写法                              | 含义             |
|---------------------------------|----------------|
| <code>const int *p</code>       | 指向常量的指针，*p 不可改 |
| <code>int *const p</code>       | 指针常量，p 不可改指向   |
| <code>const int *const p</code> | 两者都不可改         |

## 6.8 危险用法（务必避免）

| 错误        | 后果       |
|-----------|----------|
| 未初始化指针解引用 | 未定义行为、崩溃 |
| 返回局部变量地址  | 悬空指针     |
| 越界访问      | 数据损坏、崩溃  |
| 释放后继续使用   | 悬空指针     |
| 重复 free   | 堆损坏      |

## 6.9 指针速查示例

| 需求    | 代码                                                           |
|-------|--------------------------------------------------------------|
| 遍历数组  | <code>for (int *p=a; p&lt;a+n; p++) printf("%d", *p);</code> |
| 字符串遍历 | <code>while (*s) { ...; s++; }</code>                        |
| 动态分配  | <code>int *p = (int*)malloc(n * sizeof(int));</code>         |
| 释放内存  | <code>free(p); p = NULL;</code>                              |

## 6.10 指针与数组作参数的区别

| 传参方式                    | sizeof 在函数内 | 能否改指向     |
|-------------------------|-------------|-----------|
| 数组 <code>int a[]</code> | 指针大小（8/4）   | 形参是指针副本   |
| 指针 <code>int *p</code>  | 指针大小        | 改 p 不影响实参 |

获取数组长度：调用前用 `sizeof(a)/sizeof(a[0])` 计算，传入函数。

# 7 数组

## 7.1 一维数组

| 写法    | 说明        | 示例                            |
|-------|-----------|-------------------------------|
| 定义    | 类型 名[长度]; | int a[10];                    |
| 初始化   | 定义时赋初值    | int a[5] = {1,2,3,4,5};       |
| 部分初始化 | 其余为 0     | int a[5] = {1,2}; → 1,2,0,0,0 |
| 省略长度  | 由初值个数决定   | int a[] = {1,2,3}; → 长度 3     |
| 全零    | 常用写法      | int a[10] = {0};              |

## 7.2 数组访问

| 规则   | 说明              |
|------|-----------------|
| 下标   | 从0 开始           |
| 合法范围 | 0 ~ 长度-1        |
| 越界   | C 不检查，可能导致未定义行为 |

```
int a[5] = {10, 20, 30, 40, 50};
printf("%d\n", a[0]);    /* 10 */
printf("%d\n", a[2]);    /* 30 */
a[4] = 100;              /* 修改最后一个元素 */
```

## 7.3 二维数组

| 写法   | 说明          |
|------|-------------|
| 定义   | 类型 名[行][列]; |
| 初始化  | 按行赋初值       |
| 元素个数 | 行 × 列       |
| 存储   | 行优先（先存第一行）  |

```
int a[2][3] = {{1,2,3}, {4,5,6}};
printf("%d\n", a[1][2]);    /* 6 */
```

## 7.4 数组与 sizeof

| 表达式                                 | 含义（一维 int a[10]） |
|-------------------------------------|------------------|
| <code>sizeof(a)</code>              | 整个数组字节数（40）      |
| <code>sizeof(a[0])</code>           | 单个元素字节数（4）       |
| <code>sizeof(a)/sizeof(a[0])</code> | 元素个数（10）         |

## 7.5 数组作为函数参数

```
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main(void) {
    int arr[] = {1, 2, 3, 4, 5};
    printArray(arr, 5);
    return 0;
}
```

| 要点    | 说明                                               |
|-------|--------------------------------------------------|
| 传参退化  | 数组名传参变为指针                                        |
| 必须传长度 | 函数内无法 sizeof 得元素个数                               |
| 二维数组  | 需指定列数： <code>void f(int a[][4], int rows)</code> |

## 7.6 常见操作

| 操作  | 代码思路                                |
|-----|-------------------------------------|
| 遍历  | <code>for (i=0; i&lt;n; i++)</code> |
| 求和  | 累加 <code>a[i]</code>                |
| 求最大 | 设 <code>max=a[0]</code> ，逐个比较       |
| 逆序  | 首尾交换，向中间靠拢                          |
| 查找  | 线性遍历或二分（有序）                         |
| 复制  | 逐个赋值或 <code>memcpy</code>           |

## 7.7 数组与指针

| 表达式                    | 类型/含义                |
|------------------------|----------------------|
| <code>a</code>         | 指向首元素的指针             |
| <code>a + i</code>     | 第 $i$ 个元素地址          |
| <code>*(a + i)</code>  | 等价 <code>a[i]</code> |
| <code>&amp;a[i]</code> | 第 $i$ 个元素地址          |

## 7.8 字符数组 vs 字符指针

|        |                               |                              |
|--------|-------------------------------|------------------------------|
|        | <code>char s[] = "hi";</code> | <code>char *s = "hi";</code> |
| 存储     | 栈上数组，内容可改                     | 指针指向字符串常量                    |
| 修改     | <code>s[0]='H'</code> 合法      | 修改内容非法                       |
| sizeof | 数组大小                          | 指针大小                         |

# 8 字符串

## 8.1 字符串本质

| 要点  | 说明                                                |
|-----|---------------------------------------------------|
| 本质  | 以 <code>\0</code> 结尾的 <code>char</code> 数组        |
| 字面量 | <code>"Hello"</code> 含 6 字节: <code>Hello\0</code> |
| 结束符 | <code>'\0'</code> , ASCII 值为 0                    |
| 头文件 | <code>#include &lt;string.h&gt;</code>            |

## 8.2 字符串定义方式

| 写法                                 | 说明                       |
|------------------------------------|--------------------------|
| <code>char s[] = "hello";</code>   | 字符数组，可修改                 |
| <code>char s[20] = "hello";</code> | 指定大小，剩余为 <code>\0</code> |
| <code>char *s = "hello";</code>    | 指向字符串常量， <b>不可修改</b>     |
| <code>char s[20];</code>           | 未初始化，需后续赋值               |

### 8.3 字符串输入

| 函数                                 | 用法      | 特点                        |
|------------------------------------|---------|---------------------------|
| <code>scanf("%s", s)</code>        | 读无空格字符串 | 遇空白停止， <b>不安全</b> （需控制长度） |
| <code>fgets(s, size, stdin)</code> | 读一行     | 安全，含 <code>\n</code> ，推荐  |
| <code>gets(s)</code>               | 已废弃     | <b>禁止使用</b>               |

```
char s[100];
fgets(s, sizeof(s), stdin);
s[strcspn(s, "\n")] = '\0'; /* 去掉换行 */
```

### 8.4 字符串输出

| 函数                            | 格式符/用法              |
|-------------------------------|---------------------|
| <code>printf("%s", s)</code>  | 输出到 <code>\0</code> |
| <code>puts(s)</code>          | 输出并换行               |
| <code>fputs(s, stdout)</code> | 输出到指定流              |

### 8.5 string.h 常用函数

| 函数                   | 原型                                                        | 功能                        | 返回值           |
|----------------------|-----------------------------------------------------------|---------------------------|---------------|
| <code>strlen</code>  | <code>size_t strlen(const char *s)</code>                 | 求长度 (不含 <code>\0</code> ) | 字符个数          |
| <code>strcpy</code>  | <code>char *strcpy(char *dst, const char *src)</code>     | 复制                        | dst           |
| <code>strncpy</code> | <code>char *strncpy(..., size_t n)</code>                 | 最多复制 n 个                  | dst           |
| <code>strcat</code>  | <code>char *strcat(char *dst, const char *src)</code>     | 连接                        | dst           |
| <code>strcmp</code>  | <code>int strcmp(const char *a, const char *b)</code>     | 比较                        | 0 相等, >0 或 <0 |
| <code>strchr</code>  | <code>char *strchr(const char *s, int c)</code>           | 找字符                       | 首次出现位置        |
| <code>strstr</code>  | <code>char *strstr(const char *s, const char *sub)</code> | 找子串                       | 首次出现位置        |

## 8.6 字符串比较规则

| <code>strcmp</code> 返回值 | 含义           |
|-------------------------|--------------|
| <code>0</code>          | 两串相等         |
| <code>&gt; 0</code>     | 第一个更大 (按字典序) |
| <code>&lt; 0</code>     | 第一个更小        |

**注意：** 不要用 `==` 比较字符串，那是比较地址。

## 8.7 字符处理 (ctype.h)

| 函数                                                | 功能     |
|---------------------------------------------------|--------|
| <code>isalpha(c)</code>                           | 是否字母   |
| <code>isdigit(c)</code>                           | 是否数字   |
| <code>isupper(c)</code> / <code>islower(c)</code> | 是否大/小写 |
| <code>isspace(c)</code>                           | 是否空白   |
| <code>toupper(c)</code> / <code>tolower(c)</code> | 转大/小写  |

## 8.8 手动实现常用操作

| 操作  | 思路                                  |
|-----|-------------------------------------|
| 求长度 | <code>while (*s++) n++;</code>      |
| 复制  | <code>while ((*d++ = *s++));</code> |
| 比较  | 逐字符比，遇 <code>\0</code> 或不等则停        |
| 反转  | 首尾指针交换                              |

## 8.9 安全注意事项

| 问题        | 建议                                                  |
|-----------|-----------------------------------------------------|
| 缓冲区溢出     | 用 <code>strncpy</code> 、 <code>fgets</code> ，确保目标够大 |
| strcpy 越界 | 先检查 dst 容量 $\geq$ src 长度+1                          |
| 字符串常量修改   | <code>char *p = "abc"; p[0]='A';</code> 非法          |
| scanf %s  | 限制宽度： <code>scanf("%99s", s)</code>                 |

## 8.10 字符串与数字转换 (stdlib.h)

| 函数                                 | 功能                            |
|------------------------------------|-------------------------------|
| <code>atoi(s)</code>               | 字符串 $\rightarrow$ int         |
| <code>atof(s)</code>               | 字符串 $\rightarrow$ double      |
| <code>sprintf(buf, "%d", n)</code> | 数字 $\rightarrow$ 字符串 (写入 buf) |

# 9 结构体与枚举

## 9.1 结构体 struct

### 9.1.1 定义与使用

```

struct Student {
    int id;
    char name[32];
    float score;
};

struct Student s1;
s1.id = 1001;
strcpy(s1.name, "Tom");
s1.score = 90.5;

```

| 操作   | 语法                                                   |
|------|------------------------------------------------------|
| 定义变量 | <code>struct Student s;</code>                       |
| 初始化  | <code>struct Student s = {1001, "Tom", 90.5};</code> |
| 访问成员 | <code>s.id</code> 、 <code>s.name</code>              |
| 指针访问 | <code>p-&gt;id</code> 等价 <code>(*p).id</code>        |

### 9.1.2 typedef 简化

```

typedef struct {
    int x, y;
} Point;

Point p = {10, 20};  /* 无需写 struct */

```

## 9.2 结构体数组

```

struct Student st[3];
st[0].id = 1;
/* 或初始化 */
struct Student st[] = {
    {1, "A", 80},
    {2, "B", 90}
};

```

## 9.3 结构体作为函数参数

| 方式   | 特点                                         |
|------|--------------------------------------------|
| 值传递  | 复制整个结构体，大结构体效率低                            |
| 指针传递 | <code>void f(struct Student *p)</code> ，推荐 |

## 9.4 结构体对齐（了解）

| 概念     | 说明            |
|--------|---------------|
| 对齐     | 成员按一定字节边界存放   |
| sizeof | 可能大于各成员之和     |
| 原因     | CPU 访问对齐地址更高效 |

## 9.5 共用体 union

| 特点 | 说明                   |
|----|----------------------|
| 内存 | 所有成员 <b>共享</b> 同一块内存 |
| 大小 | 等于最大成员的大小            |
| 用途 | 节省空间、多种格式解释同一数据      |

```
union Data {
    int i;
    float f;
    char c;
};
union Data d;
d.i = 10;      /* 此时用 i */
d.f = 3.14f;   /* 覆盖 i 的内容 */
```

## 9.6 枚举 enum

```
enum Color { RED, GREEN, BLUE };
enum Color c = GREEN;    /* GREEN 值为 1 */
```

| 特点  | 说明                                          |
|-----|---------------------------------------------|
| 默认值 | 从 0 递增                                      |
| 可指定 | <code>enum { A=1, B, C }; → B=2, C=3</code> |
| 本质  | 整型常量                                        |

## 9.7 位域（节省空间）

```
struct Flags {
    unsigned int a : 1;    /* 占 1 位 */
    unsigned int b : 3;    /* 占 3 位 */
};
```

## 9.8 对比速查

|      | struct     | union    | enum           |
|------|------------|----------|----------------|
| 成员共存 | 是          | 否（共享内存）  | —              |
| 主要用途 | 组合不同类型数据   | 多种视图/省空间 | 命名整型常量         |
| 大小   | 各成员之和（含对齐） | 最大成员     | 通常 sizeof(int) |

## 9.9 嵌套结构体

```
typedef struct {
    int year, month, day;
} Date;

typedef struct {
    char name[32];
    Date birthday;
} Person;

Person p;
p.birthday.year = 2000;
```

## 9.10 C99 指定初始化

```
struct Student s = {
    .id = 1001,
    .name = "Amy",
    .score = 95.0
};
```

# 10 预处理

## 10.1 预处理指令概览

| 指令                                                            | 作用          |
|---------------------------------------------------------------|-------------|
| <code>#include</code>                                         | 包含头文件       |
| <code>#define</code>                                          | 宏定义         |
| <code>#undef</code>                                           | 取消宏定义       |
| <code>#if</code> / <code>#ifdef</code> / <code>#ifndef</code> | 条件编译        |
| <code>#else</code> / <code>#elif</code> / <code>#endif</code> | 条件分支        |
| <code>#pragma</code>                                          | 编译器相关（实现依赖） |
| <code>#error</code>                                           | 编译报错并输出信息   |

特点：以 `#` 开头，不需要分号结尾。

## 10.2 头文件包含

| 写法                                    | 搜索顺序        |
|---------------------------------------|-------------|
| <code>#include &lt;stdio.h&gt;</code> | 系统目录（标准库）   |
| <code>#include "myfile.h"</code>      | 当前目录 → 系统目录 |

常见标准头文件：

| 头文件                    | 内容           |
|------------------------|--------------|
| <code>stdio.h</code>   | 输入输出         |
| <code>stdlib.h</code>  | 内存分配、转换、exit |
| <code>string.h</code>  | 字符串函数        |
| <code>math.h</code>    | 数学函数         |
| <code>ctype.h</code>   | 字符处理         |
| <code>time.h</code>    | 时间日期         |
| <code>stdbool.h</code> | bool 类型（C99） |

## 10.3 宏定义 #define

### 10.3.1 常量宏

```
#define PI 3.14159
#define MAX 100
```

### 10.3.2 带参宏

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define SQUARE(x) ((x) * (x))
```

| 注意    | 说明                                                           |
|-------|--------------------------------------------------------------|
| 加括号   | 防止运算符优先级问题                                                   |
| 副作用   | 避免 <code>#define SQ(x) x*x</code> 在 <code>SQ(i++)</code> 时出错 |
| 无类型检查 | 宏只是文本替换                                                      |

### 10.3.3 宏 vs const

|      | <code>#define</code> | <code>const</code> |
|------|----------------------|--------------------|
| 处理阶段 | 预处理                  | 编译                 |
| 类型   | 无                    | 有                  |
| 调试   | 难                    | 易                  |
| 推荐   | 条件编译、简单常量            | 一般常量               |

## 10.4 条件编译

```
#define DEBUG 1

#ifdef DEBUG
    printf("调试信息\n");
#endif

#ifndef HEADER_H
#define HEADER_H
/* 头文件内容 */
#endif
```

| 指令                         | 含义             |
|----------------------------|----------------|
| <code>#ifdef MACRO</code>  | 宏已定义则编译        |
| <code>#ifndef MACRO</code> | 宏未定义则编译（头文件保护） |
| <code>#if 表达式</code>       | 表达式非 0 则编译     |
| <code>#else</code>         | 否则分支           |
| <code>#endif</code>        | 结束条件块          |

## 10.5 常用预处理技巧

| 场景    | 写法                                                                              |
|-------|---------------------------------------------------------------------------------|
| 头文件保护 | <code>#ifndef XXX_H / #define XXX_H / #endif</code>                             |
| 调试开关  | <code>#ifdef DEBUG ... #endif</code>                                            |
| 跨平台   | <code>#ifdef _WIN32 ... #else ...</code>                                        |
| 字符串化  | <code>#define STR(x) #x</code> → <code>STR(hello)</code> → <code>"hello"</code> |
| 连接符号  | <code>#define CAT(a,b) a##b</code>                                              |

## 10.6 预定义宏

| 宏                     | 含义         |
|-----------------------|------------|
| <code>__FILE__</code> | 当前源文件名     |
| <code>__LINE__</code> | 当前行号       |
| <code>__DATE__</code> | 编译日期       |
| <code>__TIME__</code> | 编译时间       |
| <code>__func__</code> | 当前函数名（C99） |

```
printf("错误: %s 第 %d 行\n", __FILE__, __LINE__);
```

## 10.7 #pragma 示例

```
#pragma once           /* 部分编译器：头文件只包含一次 */
#pragma warning(disable:4996) /* MSVC: 忽略某警告 */
```

## 10.8 预处理执行顺序

1. 处理 `#include`、展开 `#define`
2. 处理条件编译，删除不满足条件的代码
3. 将结果交给编译器

# 11 输入输出

## 11.1 标准流

| 流    | 文件指针                | 设备 |
|------|---------------------|----|
| 标准输入 | <code>stdin</code>  | 键盘 |
| 标准输出 | <code>stdout</code> | 屏幕 |
| 标准错误 | <code>stderr</code> | 屏幕 |

头文件： `#include <stdio.h>`

## 11.2 printf 格式输出

### 11.2.1 基本用法

```
printf("格式字符串", 参数1, 参数2, ...);
```

### 11.2.2 常用格式符

| 格式符                                  | 类型                | 示例                                    |
|--------------------------------------|-------------------|---------------------------------------|
| <code>%d</code> / <code>%i</code>    | int               | <code>printf("%d", 42);</code>        |
| <code>%u</code>                      | unsigned int      | <code>printf("%u", n);</code>         |
| <code>%ld</code> / <code>%lld</code> | long / long long  | <code>printf("%lld", x);</code>       |
| <code>%f</code>                      | float/double      | <code>printf("%.2f", 3.14159);</code> |
| <code>%e</code> / <code>%g</code>    | 科学计数法             | <code>printf("%e", 1.2e3);</code>     |
| <code>%c</code>                      | char              | <code>printf("%c", 'A');</code>       |
| <code>%s</code>                      | 字符串               | <code>printf("%s", "hello");</code>   |
| <code>%p</code>                      | 指针地址              | <code>printf("%p", p);</code>         |
| <code>%x</code> / <code>%o</code>    | 十六进制/八进制          | <code>printf("%x", 255);</code>       |
| <code>%%</code>                      | 输出 <code>%</code> | <code>printf("100%%");</code>         |

### 11.2.3 宽度和精度

| 写法                 | 含义           | 示例                 |
|--------------------|--------------|--------------------|
| <code>%5d</code>   | 宽度至少 5       | 右对齐                |
| <code>%-5d</code>  | 左对齐          | —                  |
| <code>%05d</code>  | 不足补 0        | <code>00123</code> |
| <code>%.2f</code>  | 保留 2 位小数     | <code>3.14</code>  |
| <code>%8.2f</code> | 总宽 8, 小数 2 位 | —                  |

### 11.3 scanf 格式输入

```
scanf("格式字符串", &变量1, &变量2);
```

| 格式符               | 类型     | 注意                        |
|-------------------|--------|---------------------------|
| <code>%d</code>   | int    | 需传地址 <code>&amp;x</code>  |
| <code>%lf</code>  | double | double 用 <code>%lf</code> |
| <code>%f</code>   | float  | —                         |
| <code>%c</code>   | char   | 注意缓冲区残留 <code>\n</code>   |
| <code>%s</code>   | 字符串    | 遇空白停止，需数组名（已是地址）          |
| <code>%99s</code> | 限制宽度   | 防止溢出                      |

### 11.3.1 常见陷阱

| 问题                         | 解决                                                 |
|----------------------------|----------------------------------------------------|
| 读 char 前残留 <code>\n</code> | <code>scanf(" %c", &amp;ch);</code> 格式前加空格         |
| 返回值未检查                     | <code>if (scanf("%d", &amp;n) != 1) ...</code>     |
| 数组不需 &                     | <code>scanf("%s", s)</code> 不是 <code>&amp;s</code> |

## 11.4 字符 I/O

| 函数                            | 功能    | 返回值         |
|-------------------------------|-------|-------------|
| <code>getchar()</code>        | 读一个字符 | int (含 EOF) |
| <code>putchar(c)</code>       | 写一个字符 | 写入字符        |
| <code>gets</code>             | 读一行   | 已废弃，勿用      |
| <code>fgets(s,n,stdin)</code> | 安全读一行 | s           |

## 11.5 格式化字符串函数

| 函数                                         | 作用        |
|--------------------------------------------|-----------|
| <code>sprintf(buf, fmt, ...)</code>        | 格式化写入字符串  |
| <code>sscanf(s, fmt, ...)</code>           | 从字符串解析    |
| <code>snprintf(buf, size, fmt, ...)</code> | 带长度限制，更安全 |

## 11.6 文件 I/O 预览（详见 12-文件操作）

| 函数                                         | 作用               |
|--------------------------------------------|------------------|
| <code>fopen</code> / <code>fclose</code>   | 打开/关闭            |
| <code>fprintf</code> / <code>fscanf</code> | 文件版 printf/scanf |
| <code>fgetc</code> / <code>fputc</code>    | 字符读写             |
| <code>fgets</code> / <code>fputs</code>    | 行读写              |

## 11.7 输入输出示例

```
#include <stdio.h>
int main(void) {
    int age;
    float height;
    char name[50];

    printf("请输入姓名 年龄 身高: ");
    scanf("%s %d %f", name, &age, &height);
    printf("姓名:%s 年龄:%d 身高:%.2f\n", name, age, height);

    return 0;
}
```

## 11.8 缓冲区与 fflush

| 概念                          | 说明             |
|-----------------------------|----------------|
| 缓冲                          | 输出可能暂存缓冲区      |
| <code>fflush(stdout)</code> | 强制刷新标准输出       |
| <code>\n</code>             | 通常触发 stdout 刷新 |

## 11.9 EOF

| 宏                | 值     | 用途         |
|------------------|-------|------------|
| <code>EOF</code> | 通常 -1 | 表示文件结束或读错误 |

```
int ch;
while ((ch = getchar()) != EOF)
    putchar(ch);
```

# 12 文件操作

头文件: #include <stdio.h>

## 12.1 文件指针

```
FILE *fp; /* 文件指针类型 */
fp = fopen("data.txt", "r"); /* 打开文件 */
/* ... 读写操作 ... */
fclose(fp); /* 关闭文件, 必须调用 */
```

## 12.2 fopen 打开模式

| 模式          | 含义    | 文件不存在 | 文件存在  |
|-------------|-------|-------|-------|
| "r"         | 只读    | 失败    | 从头读   |
| "w"         | 只写    | 创建    | 清空后写  |
| "a"         | 追加    | 创建    | 末尾追加  |
| "r+"        | 读写    | 失败    | 从头读写  |
| "w+"        | 读写    | 创建    | 清空后读写 |
| "a+"        | 读+追加  | 创建    | 末尾追加  |
| "rb" "wb" 等 | 二进制模式 | 同上    | 同上    |

返回值: 成功返回 FILE\*, 失败返回 NULL, 务必检查。

```
FILE *fp = fopen("test.txt", "r");
if (fp == NULL) {
    perror("打开失败");
    return 1;
}
```

## 12.3 字符级读写

| 函数                        | 功能                   | 返回值     |
|---------------------------|----------------------|---------|
| <code>fgetc(fp)</code>    | 读一个字符                | 字符或 EOF |
| <code>fputc(c, fp)</code> | 写一个字符                | 字符或 EOF |
| <code>getc(fp)</code>     | 同 <code>fgetc</code> | —       |
| <code>putc(c, fp)</code>  | 同 <code>fputc</code> | —       |

## 12.4 字符串/行读写

| 函数                           | 功能             |
|------------------------------|----------------|
| <code>fgets(s, n, fp)</code> | 读一行（最多 n-1 字符） |
| <code>fputs(s, fp)</code>    | 写字符串           |

## 12.5 格式化读写

| 函数                                 | 功能       |
|------------------------------------|----------|
| <code>fprintf(fp, fmt, ...)</code> | 格式化写入文件  |
| <code>fscanf(fp, fmt, ...)</code>  | 格式化从文件读取 |

```
fprintf(fp, "%d %.2f %s\n", id, score, name);
fscanf(fp, "%d %f %s", &id, &score, name);
```

## 12.6 块读写（二进制）

| 函数                                        | 功能                |
|-------------------------------------------|-------------------|
| <code>fread(ptr, size, count, fp)</code>  | 读 count 个 size 字节 |
| <code>fwrite(ptr, size, count, fp)</code> | 写 count 个 size 字节 |

```
int a[10];
fwrite(a, sizeof(int), 10, fp);
fread(a, sizeof(int), 10, fp);
```

## 12.7 文件定位

| 函数                                     | 功能     |
|----------------------------------------|--------|
| <code>fseek(fp, offset, origin)</code> | 移动文件指针 |
| <code>ftell(fp)</code>                 | 返回当前位置 |
| <code>rewind(fp)</code>                | 回到文件开头 |

**origin:** `SEEK_SET` (开头)、`SEEK_CUR` (当前)、`SEEK_END` (末尾)

## 12.8 文件状态检测

| 函数                        | 功能           |
|---------------------------|--------------|
| <code>feof(fp)</code>     | 是否到达文件末尾     |
| <code>ferror(fp)</code>   | 是否发生错误       |
| <code>clearerr(fp)</code> | 清除错误和 EOF 标志 |

## 12.9 文本文件 vs 二进制文件

|    | 文本模式                              | 二进制模式                               |
|----|-----------------------------------|-------------------------------------|
| 打开 | <code>"r"</code> <code>"w"</code> | <code>"rb"</code> <code>"wb"</code> |
| 内容 | 人类可读                              | 原始字节                                |
| 换行 | 可能转换                              | 不转换                                 |
| 适用 | 配置文件、日志                           | 结构体、图片、音频                           |

## 12.10 完整示例：复制文件

```
#include <stdio.h>
int main(void) {
    FILE *in = fopen("src.txt", "r");
    FILE *out = fopen("dst.txt", "w");
    if (!in || !out) return 1;

    int ch;
    while ((ch = fgetc(in)) != EOF)
        fputc(ch, out);

    fclose(in);
    fclose(out);
}
```

```
return 0;
}
```

## 12.11 完整示例：结构体存盘

```
typedef struct { int id; char name[20]; } Record;
Record r = {1, "Tom"};
FILE *fp = fopen("data.bin", "wb");
fwrite(&r, sizeof(Record), 1, fp);
fclose(fp);
```

## 12.12 文件操作注意事项

| 要点        | 说明                                          |
|-----------|---------------------------------------------|
| 检查 fopen  | 失败时返回 NULL                                  |
| 必须 fclose | 否则可能丢数据、泄漏资源                                |
| w 模式      | 会清空已有文件                                     |
| 路径        | Windows 可用 "C:\\\\data.txt" 或 "C:/data.txt" |
| 权限        | 确保有读写权限                                     |

# 13 动态内存

头文件：`#include <stdlib.h>`

## 13.1 为什么需要动态内存

| 对比   | 栈（自动变量/数组） | 堆（动态分配）       |
|------|------------|---------------|
| 大小   | 编译期或固定     | 运行期按需         |
| 生命周期 | 函数结束释放     | 手动 free 前一直存在 |
| 典型用途 | 局部变量       | 链表、可变长数据      |

## 13.2 四个核心函数

| 函数                               | 功能                          | 返回值            |
|----------------------------------|-----------------------------|----------------|
| <code>malloc(size)</code>        | 分配 size 字节， <b>不初始化</b>     | void*, 失败 NULL |
| <code>calloc(n, size)</code>     | 分配 n×size 字节， <b>初始化为 0</b> | void*          |
| <code>realloc(p, newSize)</code> | 调整已分配块大小                    | void*, 失败 NULL |
| <code>free(p)</code>             | 释放 p 指向的内存                  | void           |

## 13.3 基本用法

```
#include <stdlib.h>

/* 分配 10 个 int */
int *arr = (int *)malloc(10 * sizeof(int));
if (arr == NULL) {
    /* 分配失败处理 */
    return 1;
}

/* 使用 arr[0] ~ arr[9] */

free(arr);      /* 释放 */
arr = NULL;     /* 避免悬空指针 */
```

## 13.4 函数对比

|     | malloc                               | calloc                              |
|-----|--------------------------------------|-------------------------------------|
| 参数  | 总字节数                                 | 元素个数 + 元素大小                         |
| 初始化 | 不初始化（内容不确定）                          | 全部置 0                               |
| 示例  | <code>malloc(n * sizeof(int))</code> | <code>calloc(n, sizeof(int))</code> |

## 13.5 realloc 用法

```
int *p = malloc(5 * sizeof(int));
/* 需要更大空间 */
int *q = realloc(p, 10 * sizeof(int));
if (q == NULL) {
    free(p);    /* 失败时原块仍有效 */
    return 1;
}

p = q;
```

| 注意   | 说明            |
|------|---------------|
| 可能移动 | 返回地址可能与原 p 不同 |
| 失败   | 原内存块仍然有效      |
| 缩小   | 多余部分被释放       |

## 13.6 动态二维数组

```
int rows = 3, cols = 4;
int **a = (int **)malloc(rows * sizeof(int *));
for (int i = 0; i < rows; i++)
    a[i] = (int *)malloc(cols * sizeof(int));

/* 使用 a[i][j] */

for (int i = 0; i < rows; i++) free(a[i]);
free(a);
```

## 13.7 常见错误

| 错误         | 后果    | 正确做法                        |
|------------|-------|-----------------------------|
| 不检查 NULL   | 解引用崩溃 | <code>if (p == NULL)</code> |
| 忘记 free    | 内存泄漏  | 每个 malloc 对应 free           |
| 重复 free    | 堆损坏   | free 后置 NULL                |
| free 后继续使用 | 悬空指针  | free 后不再访问                  |
| 越界写入       | 堆损坏   | 严格控制在分配大小内                  |

## 13.8 内存泄漏

**定义：** 动态分配的内存未释放且无法再访问。

**预防：**

- 谁分配谁释放，或约定清晰的所有权
- 函数返回前 free 局部分配
- 复杂结构（链表）遍历释放每个结点

## 13.9 链表结点分配示例

```
typedef struct Node {
    int data;
    struct Node *next;
} Node;

Node *p = (Node *)malloc(sizeof(Node));
p->data = 10;
p->next = NULL;
/* 不用时 */
free(p);
```

## 13.10 malloc 与 sizeof

| 推荐写法                                 | 原因       |
|--------------------------------------|----------|
| <code>malloc(n * sizeof(int))</code> | 类型变更时仍正确 |
| 避免 <code>malloc(100)</code>          | 魔法数字，易错  |

## 13.11 栈 vs 堆 速查

| 操作   | 栈          | 堆             |
|------|------------|---------------|
| 分配   | 自动         | malloc/calloc |
| 释放   | 自动         | free          |
| 大小限制 | 较小（通常几 MB） | 较大（受系统限制）     |
| 速度   | 快          | 相对慢           |
| 碎片   | 无          | 可能产生          |

# 14 常用库函数

## 14.1 stdio.h — 输入输出

| 函数                                                            | 功能        |
|---------------------------------------------------------------|-----------|
| <code>printf</code> / <code>scanf</code>                      | 格式化标准 I/O |
| <code>fprintf</code> / <code>fscanf</code>                    | 格式化文件 I/O |
| <code>sprintf</code> / <code>sscanf</code>                    | 字符串格式化    |
| <code>getchar</code> / <code>putchar</code>                   | 字符 I/O    |
| <code>fgets</code> / <code>fputs</code>                       | 行 I/O     |
| <code>fopen</code> / <code>fclose</code>                      | 打开/关闭文件   |
| <code>fgetc</code> / <code>fputc</code>                       | 文件字符 I/O  |
| <code>fread</code> / <code>fwrite</code>                      | 块读写       |
| <code>fseek</code> / <code>ftell</code> / <code>rewind</code> | 文件定位      |
| <code>feof</code> / <code>ferror</code>                       | 文件状态      |
| <code>perror</code>                                           | 输出系统错误信息  |

## 14.2 stdlib.h — 通用工具

| 函数                                                                                   | 功能              | 示例                              |
|--------------------------------------------------------------------------------------|-----------------|---------------------------------|
| <code>malloc</code> / <code>calloc</code> / <code>realloc</code> / <code>free</code> | 动态内存            | 见 13-动态内存                       |
| <code>exit(status)</code>                                                            | 立即终止程序          | <code>exit(0);</code>           |
| <code>abort()</code>                                                                 | 异常终止            | —                               |
| <code>atoi(s)</code>                                                                 | 字符串→int         | <code>atoi("123")</code> → 123  |
| <code>atof(s)</code>                                                                 | 字符串→double      | —                               |
| <code>atol(s)</code>                                                                 | 字符串→long        | —                               |
| <code>rand()</code>                                                                  | 伪随机数 0~RAND_MAX | —                               |
| <code>srand(seed)</code>                                                             | 设置随机种子          | <code>srand(time(NULL));</code> |
| <code>abs(n)</code>                                                                  | 整数绝对值           | —                               |
| <code>qsort</code>                                                                   | 快速排序            | 需比较函数                           |
| <code>bsearch</code>                                                                 | 二分查找            | 数组须有序                           |
| <code>system(cmd)</code>                                                             | 执行系统命令          | <code>system("cls");</code>     |

### 14.2.1 qsort 示例

```
int cmp(const void *a, const void *b) {
    return *(int *)a - *(int *)b;
}
qsort(arr, n, sizeof(int), cmp);
```

## 14.3 string.h — 字符串

| 函数                   | 功能         |
|----------------------|------------|
| strlen(s)            | 求长度        |
| strcpy(dst, src)     | 复制         |
| strncpy(dst, src, n) | 安全复制 n 个   |
| strcat(dst, src)     | 连接         |
| strncat(dst, src, n) | 连接 n 个     |
| strcmp(a, b)         | 比较         |
| strncmp(a, b, n)     | 比较 n 个     |
| strchr(s, c)         | 找字符        |
| strrchr(s, c)        | 从后找字符      |
| strstr(s, sub)       | 找子串        |
| memcpy(dst, src, n)  | 复制 n 字节    |
| memset(p, c, n)      | 将 n 字节设为 c |
| memmove(dst, src, n) | 可重叠内存复制    |

## 14.4 ctype.h — 字符分类

| 函数                                                | 功能                                            |
|---------------------------------------------------|-----------------------------------------------|
| <code>isalpha(c)</code>                           | 是否字母                                          |
| <code>isdigit(c)</code>                           | 是否数字                                          |
| <code>isalnum(c)</code>                           | 是否字母或数字                                       |
| <code>isspace(c)</code>                           | 是否空白（空格、 <code>\t</code> 、 <code>\n</code> 等） |
| <code>isupper(c)</code> / <code>islower(c)</code> | 是否大/小写                                        |
| <code>toupper(c)</code> / <code>tolower(c)</code> | 转大/小写                                         |

## 14.5 math.h — 数学（链接时加 `-lm`）

| 函数                                                              | 功能       |
|-----------------------------------------------------------------|----------|
| <code>sqrt(x)</code>                                            | 平方根      |
| <code>pow(x, y)</code>                                          | x 的 y 次幂 |
| <code>fabs(x)</code>                                            | 浮点绝对值    |
| <code>ceil(x)</code> / <code>floor(x)</code>                    | 上取整/下取整  |
| <code>sin(x)</code> / <code>cos(x)</code> / <code>tan(x)</code> | 三角函数     |
| <code>log(x)</code> / <code>log10(x)</code>                     | 自然/常用对数  |
| <code>exp(x)</code>                                             | e 的 x 次幂 |

## 14.6 time.h — 时间

| 函数/类型                       | 功能           |
|-----------------------------|--------------|
| <code>time_t</code>         | 时间类型         |
| <code>time(NULL)</code>     | 当前时间戳（秒）     |
| <code>clock()</code>        | CPU 时钟 ticks |
| <code>CLOCKS_PER_SEC</code> | 每秒 ticks 数   |

```
#include <time.h>
srand((unsigned)time(NULL)); /* 随机种子 */
clock_t t = clock();
/* 代码 */
printf("耗时:%f秒\n", (double)(clock()-t)/CLOCKS_PER_SEC);
```

## 14.7 assert.h — 断言

```
#include <assert.h>
assert(p != NULL); /* 条件为假则 abort 并输出信息 */
```

## 14.8 stdbool.h — 布尔 (C99)

```
#include <stdbool.h>
bool flag = true; /* 等价 _Bool, 值为 0 或 1 */
```

## 14.9 按场景速查

| 我想...  | 用哪个                                        |
|--------|--------------------------------------------|
| 读整数    | <code>scanf("%d", &amp;n)</code>           |
| 读一行    | <code>fgets(s, size, stdin)</code>         |
| 字符串长度  | <code>strlen</code>                        |
| 字符串复制  | <code>strcpy</code> / <code>strncpy</code> |
| 字符串比较  | <code>strcmp</code>                        |
| 排序数组   | <code>qsort</code>                         |
| 随机数    | <code>srand</code> + <code>rand</code>     |
| 分配内存   | <code>malloc</code> + <code>free</code>    |
| 字符串转数字 | <code>atoi</code> / <code>strtol</code>    |
| 开平方    | <code>sqrt</code> ( <code>math.h</code> )  |
| 当前时间   | <code>time(NULL)</code>                    |
| 调试断言   | <code>assert</code>                        |

## 14.10 编译链接提示

| 库   | GCC 链接选项                            |
|-----|-------------------------------------|
| 数学库 | <code>gcc prog.c -o prog -lm</code> |
| 标准库 | 通常自动链接                              |

# 14.11 安全替代建议

| 不推荐                        | 推荐                                                 |
|----------------------------|----------------------------------------------------|
| <code>gets</code>          | <code>fgets</code>                                 |
| <code>strcpy</code> (未知长度) | <code>strncpy</code> / <code>snprintf</code>       |
| <code>sprintf</code> (无边界) | <code>snprintf</code>                              |
| <code>scanf("%s")</code>   | <code>scanf("%99s", s)</code> 或 <code>fgets</code> |